Glacier Quest plan de servicios

Arquitectura alineada con los labs · Preentrega · 7 de octubre de 2026

Objetivo y reglas aplicables

Glacier Quest es una pequeña plaza social con mecánicas RPG sencillas. La prioridad del proyecto es integrar servicios de nube y poder demostrar su función. Se conserva un alcance reducido: una sala, chat, inventario y duelo por turnos. No se requieren múltiples mapas, IA generativa ni un MMO masivo.

La rúbrica exige al menos siete servicios distintos, incluyendo cómputo, almacenamiento y base de datos. IAM cuenta cuando se configuran identidades o permisos específicos y se documentan. Logging y Monitoring se consideran juntos como observabilidad. GitHub Actions cuenta cuando despliega realmente a la nube. Los productos Firebase no sustituyen servicios contabilizados.

Por indicación del equipo, Firestore queda excluido del proyecto, aunque exista una mención más amplia en las reglas escritas. Se usará PostgreSQL en Cloud SQL. No se propone Firebase Auth, Firebase Storage ni Firebase Hosting como parte del conteo.

El documento general fija M2 para el 9 de octubre de 2026 con diagrama, cuenta/proyecto configurados y al menos dos servicios desplegados. Esta preentrega puede mostrar avance local y plan, pero si se evalúa formalmente como M2, esos dos servicios activos siguen siendo un requisito pendiente. La final figura para el 2 de noviembre, con URL, PDF, video de cinco a siete minutos y ZIP. No se declaran completadas etapas que aún no se ejecutaron.

Correspondencia con los laboratorios

La comparación se basa en los títulos facilitados por el equipo, no en el contenido de las guías. «Explícito» significa que el título menciona el tema o servicio. «Inferido» significa que encaja en esa materia, pero el título no permite confirmar la herramienta exacta utilizada en clase.

Componente del proyecto	Laboratorios relacionados	Conclusión
IAM	Lab 2 IAM / VPC	Explícito; conservar
Cloud Storage	Lab 5 Cloud Storage	Explícito; conservar
Cloud SQL PostgreSQL	Lab 6 Cloud SQL	Servicio explícito; confirmar motor usado en clase
Cloud Run	Labs 7, 9 y 10 Serverless; 16 Hosting	Categoría compatible; Cloud Run no aparece por nombre
Logging y Monitoring	Lab 13 Observabilidad	Tema explícito; productos exactos no confirmados
GitHub Actions	Labs 14 DevOps y 15 Deployment	Tema compatible; Actions no aparece por nombre
Pub/Sub	Lab 18 Pub/Sub	Explícito; añadido al plan principal, pendiente de implementar
Terraform	Lab 20 Terraform	Explícito; herramienta de IaC, no otro servicio contado

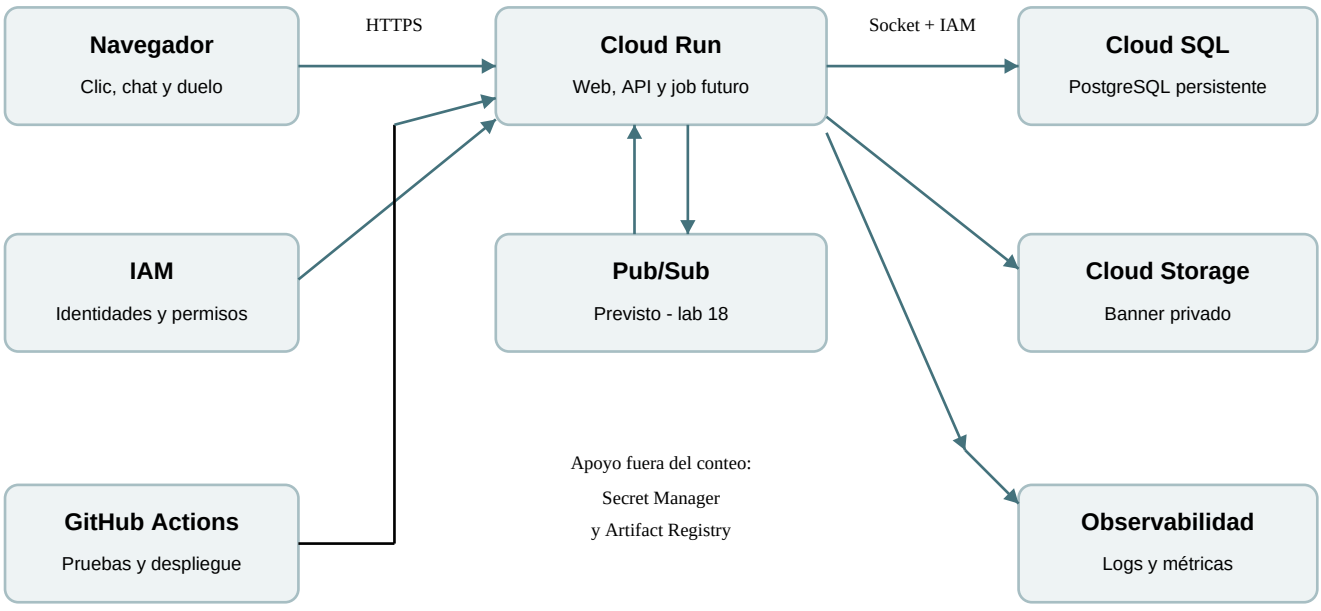
Componente del proyecto	Laboratorios relacionados	Conclusión
Secret Manager	No aparece por nombre	Apoyo técnico de seguridad; fuera del conteo de siete
Artifact Registry	Podría formar parte de Deployment o Kubernetes	No confirmado; apoyo para imágenes, fuera del conteo
Workload Identity Federation	Podría formar parte de IAM o DevOps	Técnica de autenticación dentro de IAM, no servicio extra
Docker, Node.js y Express	No aparecen por nombre	Herramientas de desarrollo; no se cuentan como servicios cloud

El plan principal pasa de ocho servicios candidatos a siete seleccionados: Cloud Run, Cloud SQL, Cloud Storage, IAM, Observabilidad, CI/CD con GitHub Actions y Pub/Sub. Logging y Monitoring siguen contando juntos. Secret Manager y Artifact Registry permanecen como componentes de apoyo porque resuelven secretos y almacenamiento de imágenes de la implementación preparada; el cumplimiento del mínimo no dependerá de contarlos.

No se cambia Cloud Run por GKE solo porque hay más labs de Kubernetes. Para esta sala pequeña, ese cambio añadiría operación de clústeres sin resolver una necesidad de la preentrega. Los labs 4 de Load balancing, 8/10/12/17 de Kubernetes o GKE, 11 de Networking y 19 de GenAI son temas disponibles, no una exigencia de utilizar todos. El lab 3 de Compute es una alternativa de cómputo; los labs 1 y 2 aportan fundamentos y configuración. Se conserva HTTPS nativo sin balanceador propio y no se incorpora IA generativa. Terraform se aprovechará para el bonus cuando corresponda.

La relación de Cloud Run y Actions con Serverless y DevOps es una inferencia, no una afirmación sobre las guías. Las reglas generales del proyecto sí los permiten. Si las guías concretas usan otras herramientas, se podrá ajustar esa elección con esa información; no es necesario sustituirla basándose solo en un título genérico.

Arquitectura prevista



Arquitectura prevista. La demo actual es local; recursos de nube pendientes.

El navegador se conectará a la URL HTTPS de Cloud Run. El mismo servicio entregará la interfaz y resolverá las acciones del juego. Solo el servidor accederá a Cloud SQL y al bucket privado. La cuenta runtime tendrá permisos específicos; Secret Manager entregará la clave de sesión, el código de clase y la contraseña de la base. GitHub Actions enviará imágenes al registro y actualizará Cloud Run mediante identidad federada. Pub/Sub se integrará después para procesar eventos de victoria y mostrar un historial de actividad; ese circuito todavía no está implementado.

Cloud Run ya proporciona una URL estable HTTPS del tipo run.app. Para que los compañeros entren más adelante no se necesita comprar un dominio, reservar una IP estática para la web ni contratar un balanceador. Durante la preentrega basta localhost. En una misma red local también podrían usar la dirección privada de la máquina y su puerto, sujeto al firewall; eso no es un despliegue cloud y no se configuró en esta entrega.

La instancia Cloud SQL tendrá su propia dirección técnica si se habilita IPv4, pero los jugadores nunca se conectan directamente a ella. La API usa la integración Cloud SQL de Cloud Run y un socket, con IAM y autenticación PostgreSQL. No se habilitan redes autorizadas para acceso directo de internet.

Matriz de servicios

Servicio	Uso concreto	Evidencia requerida
Cloud Run	Web y API Express en contenedor; escala 0 a 2	URL HTTPS, revisión activa y respuesta de salud
Cloud SQL PostgreSQL	Inventario, monedas, salud, turnos, chat y presencia	Recolección persistente; filas y transacción de turno
Cloud Storage	Banner privado de expedición que consume la API	Objeto en bucket y recurso mostrado en juego
IAM	Cuentas runtime y deploy separadas, roles limitados y OIDC	Roles por recurso y restricciones de GitHub
Cloud Logging y Monitoring	Peticiones JSON y panel de latencia/tráfico	RequestId visible en logs y métricas reales
GitHub Actions	Pruebas, imagen y despliegue hacia GCP	Ejecución exitosa con nueva revisión
Pub/Sub	Eventos de victoria y procesamiento diferido del historial	Mensaje publicado, recibido y confirmado; actividad guardada una sola vez

Son siete servicios del plan principal. Pub/Sub está previsto para una siguiente etapa: todavía no existe un publicador, consumidor ni recurso Terraform de Pub/Sub en el código entregado. Hasta completar su integración y comprobar el resto, no se debe afirmar que el proyecto ya cumple los siete servicios. Secret Manager y Artifact Registry son apoyo técnico y no forman parte de este conteo.

Qué hará cada servicio

Cloud Run

Recibirá las acciones de movimiento, chat, recolección y combate. Entregará también HTML, CSS y JavaScript, evitando dos dominios y configuración CORS. El contenedor escucha el puerto que indique el entorno y expone /health. Se configuran cero instancias mínimas, máximo dos y cinco conexiones SQL por instancia. El máximo no es un tope de gasto; siguen contando tiempo de uso y otros servicios.

Cloud SQL PostgreSQL

Se elige para conservar una base relacional permitida y cercana al plan original. Las tablas gq_players, gq_presence y gq_chat separan estado persistente, presencia y conversación. La lógica usa parámetros SQL y transacciones con bloqueo por jugador para evitar turnos repetidos y recompensas duplicadas. El administrador aplica el esquema; el usuario runtime solo recibe permisos de datos.

Para empezar se propone PostgreSQL 16, edición Enterprise, db-f1-micro, zona única y 10 GiB SSD. Es una instancia pequeña para clase; no ofrece las prestaciones ni disponibilidad de una configuración de producción. No hay alta disponibilidad, backups automáticos ni recuperación a un punto en el tiempo en esta configuración de demo. Antes de guardar información valiosa deberán habilitar respaldos o exportar y comprobar restauración.

Cloud Storage

Contendrá el SVG de las montañas que aparece en el diario del juego. El bucket será privado, con acceso público prevenido. La API descargará el objeto usando su cuenta de servicio y lo entregará al navegador. Esta ruta permite demostrar almacenamiento sin pagar Cloud CDN ni balanceador. La caché por instancia dura cinco minutos; el botón de comprobación vuelve a consultar el objeto.

IAM

La identidad runtime tendrá Cloud SQL Client, lectura del bucket y acceso únicamente a los tres secretos. El usuario SQL tiene permisos aparte: IAM no sustituye las autorizaciones de PostgreSQL. La identidad deploy escribirá imágenes y el recurso visual, actualizará el servicio existente y podrá actuar como runtime para desplegarlo. No recibe Owner ni Editor.

GitHub se autentica con Workload Identity Federation. El proveedor restringe el ID numérico del repositorio, el propietario y la rama main. El equipo debe proteger esa rama y revisar cambios de workflows. No se necesita subir una clave JSON permanente a GitHub.

Secret Manager como apoyo técnico

Separa del código la clave HMAC, el código compartido de clase y la contraseña SQL. Cloud Run recibe versiones explícitas; rotarlas requiere crear versión y actualizar la revisión. Cambiar la clave de firma invalida sesiones anteriores. Nunca deben mostrarse valores en el video, PDF, logs o capturas. Las versiones de secreto se cargan fuera de Terraform para no guardar los valores en su estado.

Observabilidad

La API escribe logs JSON con identificador de petición, ruta, estado y duración. No registra nombres, mensajes ni tokens. Cloud Logging recoge esos logs al desplegar. Terraform prepara un panel de peticiones y latencia con métricas nativas de Cloud Run. Se debe ejecutar tráfico real y buscar una petición para probar la integración. No se programan sondas permanentes ni alertas métricas de pago por defecto.

GitHub Actions con Artifact Registry de apoyo

La validación ejecuta pruebas, build Docker y comprobación Terraform. El despliegue se lanza manualmente desde main para no publicar cada experimento. Utiliza OIDC, sube una imagen con el SHA del commit, actualiza el banner y Cloud Run, y comprueba /health con base postgres. El registro conserva tres imágenes recientes y elimina versiones viejas según la política. Un workflow escrito o una imagen local no prueba CI/CD: falta ejecutarlo en el repositorio real.

Pub/Sub del lab 18

Se añadirá un historial de victorias procesado de forma asíncrona. La recompensa del combate seguirá resolviéndose en la transacción SQL; Pub/Sub no decidirá si el jugador ganó ni otorgará monedas otra vez. El mensaje solo permitirá actualizar el historial o las estadísticas de actividad fuera del camino inmediato de respuesta del juego.

Flujo previsto: al terminar una victoria, la misma transacción guardará el estado del jugador y una fila pendiente en una tabla de eventos de salida. Un proceso finito ejecutado como Cloud Run Job publicará los eventos pendientes en el topic glacierquest-events. Una suscripción pull glacierquest-activity entregará los mensajes al consumidor, que insertará la actividad y confirmará la recepción después de guardarla. El publicador y consumidor podrán ejecutarse por fases en el mismo job de demostración. Es otra carga de Cloud Run, no un servicio adicional para aumentar el conteo. Se iniciará manualmente durante el ensayo, sin añadir un planificador.

El identificador del evento será estable, por ejemplo victory seguido del ID de batalla. El historial tendrá una restricción única sobre ese identificador. Pub/Sub puede entregar mensajes más de una vez; si se repite uno, el consumidor conservará un solo registro y luego confirmará el mensaje. La tabla de salida permitirá reintentar cuando falle la publicación después de guardar el combate. Un topic vacío o un mensaje enviado manualmente desde consola no demostrarán por sí solos integración con el juego.

Implementación pendiente: tablas de salida e historial, inserción del evento en la transacción de victoria, SDK oficial de Pub/Sub, topic y suscripción en Terraform, job, permisos publisher/subscriber limitados a esos recursos y vista de actividad. La prueba de aceptación será ganar un duelo, ejecutar el job, ver una actividad y reenviar el mismo evento sin duplicarla; también probar una publicación fallida y su recuperación. Hasta entonces, esta sección es diseño, no funcionalidad disponible.

Etapas de trabajo

Etapa	Trabajo	Criterio de cierre
Ahora	Demo local, reglas del combate, documentos y diseño	Juego inicia; se explica el rol futuro de cada servicio
Siguiente preparación	PostgreSQL local con Compose, migraciones y pruebas SQL	Datos sobreviven al reinicio; turnos concurrentes no duplican premio
Primera nube	Proyecto/facturación; desplegar y demostrar servicios	Evidencia de dos servicios como mínimo si se evalúa M2
Integración de aplicación	Cloud SQL, Storage, secretos e IAM; imagen y Cloud Run	Web por HTTPS con inventario persistente y recurso del bucket
Entrega operativa	Actions, registro de apoyo y observabilidad	Commit desplegado y petición trazable en logs/panel
Integración del lab 18	Pub/Sub, job de actividad, eventos de salida e historial SQL	Victoria publicada y consumida; reintentos sin duplicar actividad
Antes de la final	Ensayo con varias personas, costos reales, video y evidencias	Siete o más servicios comprobados y documentación final

Para el checkpoint se priorizará evidencia de servicios del plan principal. Cloud Run conectado a Cloud SQL demuestra cómputo y base de datos de forma clara; Storage puede añadirse mostrando el recurso privado. No se usará Artifact Registry como sustituto automático de los dos servicios exigidos, ya que ahora figura como apoyo. No se fuerza un despliegue sin cuenta ni presupuesto disponibles.

Despliegue y configuración

La guía ejecutable docs/deployment-steps.md acompaña este informe. El flujo es: preparar proyecto dedicado y presupuesto; revisar Terraform con las etapas desactivadas; crear recursos de base; generar secretos; habilitar Cloud SQL cuando se vaya a usar; crear usuario y esquema; construir y subir imagen; desplegar Cloud Run; configurar variables de Actions, implementar la etapa de Pub/Sub y finalmente reunir evidencias.

Por defecto enable_database=false y deploy_app=false. Un apply sí puede crear otros recursos de preparación: no debe ejecutarse sin revisar el plan. Habilitar ambos indicadores provisiona los componentes facturables. No se aplica Terraform desde la demo local ni se intenta desplegar automáticamente al abrir el juego.

Guardar terraform.tfstate fuera de Git y acordar quién ejecuta cambios. La versión entregada usa estado local; antes de trabajo simultáneo se recomienda migrarlo a un backend protegido. Los recursos existentes deben importarse o conciliarse, no recrearse por suposición. El proyecto no migra datos de una base anterior ni los elimina.

Estimación económica

Región de referencia: us-central1, USD, precios consultados en octubre de 2026. Son estimaciones, no garantía de factura. No se descuentan créditos promocionales. La preentrega en modo memoria o PostgreSQL local no contrata nube; el costo cloud es cero mientras no se provisionen recursos.

Componente	Supuesto de planificación	Aproximación
Cloud SQL cómputo	db-f1-micro a USD 0,0105 por hora; 730 horas	USD 7,67 por mes
Cloud SQL SSD	10 GiB a referencia USD 0,17/GiB-mes	USD 1,70 por mes
Cloud SQL encendido	Suma anterior sin extras	USD 9,37 por mes
IPv4 ociosa	Si aplica tarifa USD 0,01/h al detener instancia	Hasta USD 7,30 por mes de inactividad
Cloud Run	Uso académico breve, min 0, cuotas disponibles	Cómputo posiblemente dentro del cupo; red aparte
Storage y secretos	Un SVG y tres versiones, poco acceso	Cercano a cero dentro de cuotas
Registro	Ejemplo 1 GiB-mes con 0,5 GiB gratuitos	Aproximadamente USD 0,05 extra
Observabilidad y Actions	Poco log, métricas nativas, minutos disponibles	Dentro de cuota bajo esos supuestos
Pub/Sub previsto	Pocos eventos de victoria, una suscripción pull y consumo durante el ensayo	Entrega básica dentro de 10 GiB/mes gratuitos si hay cupo disponible

El cupo de Pub/Sub es compartido por cuenta de facturación y corresponde a entrega básica. El job consumidor también usa cómputo, consultas SQL y logs; la retención y el tráfico pueden generar cargos adicionales. La estimación supone eventos pequeños y ejecución breve, no polling permanente.

Presupuesto prudente al mantener SQL todo el mes: USD 12-15, sujeto a región, red, cuota disponible y consumo real. Esta cifra reemplaza la estimación anterior basada en una base distinta. Alertas sugeridas: USD 5, 10 y 15; no detienen automáticamente los servicios.

Detener Cloud SQL no equivale a eliminar todos los cargos. Continúan disco y posiblemente IPv4 ociosa. Por ejemplo, once horas encendida y 719 detenida darían unos USD 0,12 de cómputo + 1,70 de disco + 7,19 de IPv4 si aplica esa tarifa. Conviene revisar la factura y no prometer un ahorro grande solo por detenerla. Otra alternativa para una demo desechable es crear la instancia cerca de la entrega y retirarla después de exportar y verificar lo necesario; borrar infraestructura pierde datos y requiere una decisión explícita del equipo.

Se evita Cloud CDN y el balanceador externo: una regla a USD 0,025/h equivale a USD 18,25 por 730 horas, antes de otros cargos. La URL nativa de Cloud Run resuelve el acceso web con menos componentes. No se incluye Vertex AI porque no es necesario para el mínimo de servicios ni para explicar el enemigo.

Seguridad y limitaciones

La aplicación valida sesiones firmadas y no confía en playerId del cliente. Los objetos se recogen una vez y las mutaciones SQL bloquean la fila de estado. El chat se muestra como texto. Hay límite de solicitudes por instancia, pero no una protección distribuida completa. El código de clase reduce acceso casual; no equivale a una cuenta recuperable.

No se valida velocidad de movimiento: las coordenadas todavía pueden manipularse. La sala muestra como máximo cuarenta presencias; treinta jugadores es un objetivo futuro que debe probarse en nube. Los datos de chat y jugadores necesitan limpieza al cerrar el curso. Las sesiones se conservan al recargar la misma pestaña; al cerrar o perder el token no hay recuperación. Estas limitaciones son aceptables como alcance explícito de una preentrega, no deben anunciarse como producción terminada.

Evidencias para la entrega

Cada evidencia debe incluir fecha, commit y estado real. Guardar URL de Cloud Run y /health; inventario antes y después de una acción; objeto en bucket; referencias de secretos y roles; ejecución de Actions; imagen con digest; petición en Logging y panel con datos. Para Pub/Sub, mostrar una victoria publicada, recepción/confirmación, registro de actividad y prueba de duplicado. No incluir contraseñas, tokens ni claves.

El video final puede dedicar un minuto a propósito y diagrama; dos a movimiento, chat, objeto y duelo; dos a consola, pipeline y trazabilidad; y uno a costos y límites. Por ahora, la presentación debe cerrar indicando qué servicios están preparados y cuáles faltan por desplegar, sin simular un estado cloud exitoso.

Fuentes

Reglas: _2026__Course_Project.docx. Contexto original: Glacier_Quest_Project_Playbook.docx. La exclusión de Firestore sigue la aclaración del equipo. El enlace Roastfolio se conserva como referencia de inspiración; no se pudo consultar su contenido y no se usa como evidencia técnica.

• Cloud SQL precios: https://cloud.google.com/sql/pricing

• Cloud Run precios: https://cloud.google.com/run/pricing

• URL HTTPS nativa: https://docs.cloud.google.com/run/docs/triggering/https-request

• Cloud Run y Cloud SQL: https://docs.cloud.google.com/sql/docs/postgres/connect-run

• Cloud Storage: https://cloud.google.com/storage/pricing

• Secret Manager: https://cloud.google.com/secret-manager/pricing

• Observabilidad: https://cloud.google.com/products/observability/pricing

• Artifact Registry: https://cloud.google.com/artifact-registry/pricing

• Balanceador: https://cloud.google.com/load-balancing/pricing

• GitHub Actions: https://docs.github.com/en/billing/concepts/product-billing/github-actions

• Pub/Sub introducción: https://docs.cloud.google.com/pubsub/docs/overview

• Pub/Sub entrega y suscripciones: https://docs.cloud.google.com/pubsub/docs/subscription-overview

• Pub/Sub precios: https://cloud.google.com/pubsub/pricing

• Calculadora: https://cloud.google.com/products/calculator